NIIT UNIVERSITY
THE UNIVERSITY OF THE FUTURE

Capstone Project II

Team No: 8

AI-BASED CYBER THREAT DETECTION AND ALERTING SYSTEM

PacketGuard AI

Software Requirements Specification

S NO	NAME	ENROLMENT NO
1	Shraddha Mandal	BT23GCS100
2	Anjali Nair	BT23GCS097
3	Ayushi Nawal	BT23GCS520
4	Vinisha Shah	BT23GCS202

Faculty Mentor: Mr. Manish Hurkat

Date: 20/09/2026

Table of Contents

1. Introduction
   1.2 Purpose
   1.2 Scope
   1.3 Definitions, Acronyms and Abbreviations
   1.4 References
   1.5 Overview

2. Overall Description
   2.1 Product Perspective
   2.2 Product Functions
   2.3 User Characteristics
   2.4 Constraints
   2.5 Assumptions and Dependencies
   2.6 Apportioning of Requirements

3. Specific Requirements
   3.1 External Interface Requirements
   3.2 Functional Requirements
   3.3 Performance Requirements
   3.4 Logical Database Requirements
   3.5 Design Constraints
   3.6 Software System Attributes
   3.7 Functional Requirement Organization
   3.8 Additional Requirements

4. Change Management Process
   4.1 Change Request Process
   4.2 Requirement review and Approval
   4.3 Requirement Version Control

5. Document Approvals
   5.1 Approvals and Sign-off

6. Supporting Information
   6.1 System Context Diagram
   6.2 Use Case Diagrams
   6.3 Data Flow Diagram (Level 1)
   6.4 ER Diagram
   6.5 Requirement Traceability Matrix
   6.6 Sample API / Input-Output Formats

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements of **PacketGuard AI**, an AI-based real-time network intrusion detection and automated alerting platform. It states what the system must do, the constraints it works under, and the quality it must achieve. It does not prescribe detailed design, which is covered in the Software Design Document (SDD).

The intended audience is:

- the project team (developers, ML engineers, testers)
- the capstone faculty and reviewers
- future maintainers of the system

1.2 Scope

PacketGuard AI is a web-based network security platform for two main users: security analysts and administrators.

The system will:

- register and authenticate users with role-based access
- capture and stream network traffic as flow-level events for analysis
- extract flow features and detect malicious traffic using a hybrid ML engine (supervised classification and unsupervised anomaly detection)
- generate an explainable severity score and top contributing features for every alert
- let analysts review, acknowledge, resolve or escalate alerts, and approve a simulated IP block
- enrich alerts with third-party threat-intelligence lookups
- notify analysts of critical alerts by email/Slack in real time
- record every AI verdict and analyst decision in an audit log

The system will not:

- automatically block, quarantine or otherwise act on network traffic without an analyst's explicit approval
- replace a commercial SIEM/SOAR platform or a licensed IDS/IPS appliance
- capture, store or analyze traffic content beyond flow-level metadata (no packet payload content inspection beyond header/flag-derived features)

Main objectives:

- reduce the time analysts spend manually triaging network traffic
- catch both known attack signatures and previously unseen anomalous behavior
- keep every AI decision transparent, explainable and auditable
- deliver the system as a secure, containerized and horizontally scalable application

Expected benefits: faster threat triage for analysts, fewer missed zero-day-like attacks through the anomaly model, and a consistent, auditable detection-and-response process.

1.3 Definitions, Acronyms and Abbreviations

- **AI:** Artificial Intelligence
- **ML:** Machine Learning
- **IDS / IPS:** Intrusion Detection System / Intrusion Prevention System
- **API:** Application Programming Interface
- **REST:** a style of web API using HTTP requests
- **JWT:** JSON Web Token, used to keep users securely logged in
- **RBAC:** Role-Based Access Control
- **MERN:** MongoDB, Express.js, React, Node.js
- **Flow:** a summarized network conversation between two endpoints (5-tuple: source/destination IP, source/destination port, protocol) over a time window
- **Feature extraction:** converting raw packets into a structured numeric flow record
- **Supervised model:** an ML model trained on labeled attack/benign traffic to name the attack family
- **Anomaly model:** an unsupervised ML model trained only on benign traffic that flags statistically unusual flows
- **Verdict fusion:** the rule combining the supervised and anomaly model outputs into one benign/malicious decision
- **SHAP:** SHapley Additive exPlanations, a method for explaining individual ML predictions
- **Severity score:** a value indicating how serious a flagged flow is (low/medium/high/critical)
- **Alert:** a flow that the detection engine has flagged as suspicious or malicious
- **Block rule:** a simulated IPS action recording that an IP should be treated as blocked
- **Threat intelligence enrichment:** looking up a suspicious IP against an external reputation database
- **Big-data streaming layer:** the message-queue backbone (Redpanda, Kafka-API-compatible) that carries flow events from capture to inference
- **Audit log:** a permanent record of important system and analyst actions
- **CI/CD:** Continuous Integration and Continuous Delivery
- **SRS / SDD:** Software Requirements Specification / Software Design Document

1.4 References

1. IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
2. ISO/IEC/IEEE 29148:2018, Requirements Engineering
3. Capstone project guidelines and SRS template provided by the faculty
4. OWASP Top 10 (web application security risks)
5. CIC-IDS2017 and UNSW-NB15 dataset documentation (Canadian Institute for Cybersecurity; UNSW ACCS)
6. Official documentation of React, Node.js, Express.js, MongoDB, FastAPI, Docker and Kubernetes
7. Documentation of the ML libraries used (scikit-learn, XGBoost, SHAP)
8. PacketGuard AI capstone roadmap document (internal project reference)

1.5 Overview

This document has six sections:

- **Section 1 (Introduction):** the purpose, scope, terms and references.
- **Section 2 (Overall Description):** the system's context, interfaces, main functions, users, constraints, assumptions and release apportioning.
- **Section 3 (Specific Requirements):** the detailed requirements: external interfaces, functional requirements, performance, database, design constraints, quality attributes and requirement organization.
- **Section 4 (Change Management Process):** how requirement changes are handled.
- **Section 5 (Document Approvals):** sign-off.
- **Section 6 (Supporting Information):** appendices with diagrams, the traceability matrix and sample formats.

Readers who want a quick understanding should read Section 2. Developers and testers should focus on Section 3.

2. Overall Description

2.1 Product Perspective

PacketGuard AI is a new, self-contained web application. It is not part of a larger system. Seen from outside, it takes network traffic as input, and it uses a few external services (email/Slack, a threat-intelligence provider, and its own internal ML inference service).

External actors and systems:

- Security Analyst and Administrator (human users)
- Network sensor / packet source (live interface or replayed capture)
- Streaming platform (Redpanda, Kafka-API-compatible)
- ML inference service (internal microservice)
- Threat intelligence provider (e.g. AbuseIPDB)
- Email/Slack notification service

2.1.1 System Interfaces

The system exchanges information with these external systems:

- **Streaming platform:** the sniffer publishes flow-feature events; the ML inference service consumes them.
- **Threat intelligence provider:** the system sends a suspicious IP and receives a reputation score and metadata.
- **Email/Slack:** the system sends critical-severity alert notifications.

2.1.2 User Interfaces

The system provides a responsive web interface with role-specific screens:

- **Public:** landing page, sign in, sign up.
- **Security Analyst:** dashboard, live alert feed, alert detail (packet payload viewer, SHAP explanation), block-approval workflow, notifications, settings.
- **Administrator:** dashboard, user management, block-rule management, detection-rule/threshold configuration, audit logs.
- **Common:** help center, about, terms and privacy pages.

2.1.3 Hardware Interfaces

The system does not control specialized security hardware directly. It reads traffic from a standard network interface (live capture) or from replayed `.pcap` files; no dedicated appliance is required for the capstone deployment.

2.1.4 Software Interfaces

- Web browsers: Chrome, Edge, Firefox and Safari (latest versions)
- Packet capture libraries: Scapy, pyshark/tshark
- Streaming client: Kafka-API-compatible client libraries against Redpanda
- Threat-intelligence REST API (e.g. AbuseIPDB)
- Email/Slack webhook API

2.1.5 Communications Interfaces

- HTTPS (TLS 1.2 or later): all communication between users' browsers and the system
- REST over HTTP/JSON and WebSocket (Socket.io): between frontend and backend
- Kafka-protocol TCP: between the sniffer, streaming platform and ML inference service

2.1.6 Memory and Storage Constraints

No externally imposed memory constraint exists. The system needs enough storage for flow/alert records (MongoDB) and relational records (SQLite via Prisma), and storage must be expandable as traffic volume grows.

2.1.7 Operational Requirements

- **Normal operation:** traffic is captured continuously (or replayed for demo) and alerts appear on the dashboard in real time.
- **Monitoring:** the system logs errors, authentication events and ML inference results; each service exposes a health-check endpoint.
- **Backup and recovery:** the databases are backed up regularly and can be restored.
- **Startup and shutdown:** all components start and stop as containers, with health checks reporting readiness.
- **Degraded mode:** if the ML inference service is unavailable, the dashboard and analyst workflow remain usable; flow events queue in the streaming layer and are processed once the service recovers.

2.1.8 Site Adaptation Requirements

Not applicable. PacketGuard AI is a web application with no physical site installation. Deployment settings (URLs, keys, database addresses, capture interface) are set through environment configuration.

2.2 Product Functions

At a customer level, the system provides:

1. **Authentication:** registration, login, roles, password reset.
2. **Traffic ingestion:** live or replayed packet capture, flow feature extraction, streaming publication.
3. **Hybrid detection:** supervised attack-family classification plus unsupervised anomaly detection, fused into one verdict.
4. **Explainability:** SHAP-based top contributing features shown per alert.
5. **Alert lifecycle management:** new → acknowledged → resolved / false positive / blocked.
6. **Simulated response:** analyst-approved IP block rules (a lightweight IPS action).
7. **Threat-intel enrichment:** reputation lookup for suspicious IPs.
8. **Notifications:** real-time in-app updates and email/Slack alerts for critical severity.
9. **Dashboards:** live severity charts, alert counts, pipeline status.
10. **Reporting and analytics:** detection metrics, alert trends, model performance.
11. **Administration:** user management, block-rule and threshold configuration.
12. **Audit logging:** every AI verdict and analyst decision recorded immutably.

2.3 User Characteristics

User	Description	Technical knowledge
Security Analyst	Monitors the live dashboard, investigates alerts, and approves or rejects response actions	Basic security/networking familiarity; needs no ML knowledge
Administrator	Manages analyst accounts, block rules, detection thresholds, and audit logs	Moderate technical knowledge

2.4 Constraints

- **Technology:** the system must use the MERN stack for the web application.
- **Deployment:** Docker is required, and Kubernetes is used for final deployment.
- **Process:** Jira for project management, Git from the start, and CI/CD via Jenkins.
- **Team:** [Team Number], working under the guidance of faculty mentor [Faculty Mentor Name].
- **Time:** limited to the capstone timeline (3 weeks of active development).
- **ML cost:** models must run within locally hosted, open-source compute — no paid inference API is required.
- **Traffic legality:** only lawfully captured or publicly licensed sample/replayed traffic (e.g. CIC-IDS2017) is used; no unauthorized network monitoring.
- **Human control:** the system must never block, quarantine or otherwise act on traffic without explicit analyst approval.
- **Network:** users need an internet connection for the web dashboard; packet capture may run on an isolated lab network.

2.5 Assumptions and Dependencies

Assumptions:

- Traffic used for the capstone demo is either replayed from public benchmark datasets (CIC-IDS2017, UNSW-NB15) or captured on an isolated lab network with permission.
- Analysts have a modern browser and internet connection.
- The detection dataset's feature schema is compatible with the live feature extractor's output.

Dependencies:

- Availability of the streaming platform (Redpanda) and the ML inference service.
- Availability of the threat-intelligence provider's free-tier API.
- Availability of an email/Slack webhook for notifications.
- A container registry and Kubernetes environment for deployment.

2.6 Apportioning of Requirements

The current release is planned to include every feature in this SRS, developed in phases:

Phase	Content
Phase 1 (Week 1)	Repo scaffold, streaming pipeline, feature extraction, baseline supervised + anomaly models
Phase 2 (Week 1–2)	ML inference service, backend API, database schema, authentication
Phase 3 (Week 2)	Real-time frontend dashboard, SHAP explainability panel, alert lifecycle workflow
Phase 4 (Week 2–3)	Notifications, threat-intel enrichment, reporting/analytics, administration
Phase 5 (Week 3)	Containerization, CI/CD, Kubernetes deployment, testing, evaluation, documentation

Future versions (not in the current release): live IPS enforcement (actual packet blocking, not simulated), multi-sensor distributed deployment, native mobile app, and support for encrypted-traffic metadata analysis.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interface Requirements

- **UI-01:** The system shall provide a responsive web interface that works on screens from 360 px (mobile) to 1920 px (desktop).
- **UI-02:** The system shall show role-specific navigation after login: Analyst — Dashboard, Live Alerts, Alert Detail, Block Rules, Notifications, Settings; Administrator — Dashboard, Users, Detection Config, Block Rules, Audit Logs.
- **UI-03:** The system shall validate every form field on screen and show a clear message next to invalid fields.
- **UI-04:** The system shall show a loading indicator for any operation longer than 1 second.
- **UI-05:** The system shall show clear error messages that do not expose technical details or sensitive data.
- **UI-06:** The system shall show, on every alert, a notice that the severity/verdict is a recommendation and the final block decision belongs to the analyst.
- **UI-07:** The system shall redirect unauthenticated users to the login page and shall not display pages of another role.

3.1.2 Network Traffic Interface

- **Source:** a live network interface or a replayed `.pcap` capture file.
- **Valid input:** IPv4/IPv6 packets over TCP, UDP or ICMP.
- **Output to the system:** a flow-level feature record (12 core features, matching the training feature schema) published to the streaming layer.
- **Timing:** flow features shall be published within 2 seconds of flow completion.
- **Invalid input:** malformed packets shall be dropped and logged without stopping the capture pipeline.

3.1.3 ML Inference Service Interface

- **Input to the inference service:** a flow-feature JSON event consumed from the streaming topic.
- **Output from the inference service (JSON):** supervised label, supervised confidence, anomaly score, verdict (benign/malicious), severity, and the top-3 SHAP-ranked contributing features.
- **Timing:** a flow shall be scored within 15 seconds of being published to the stream; requests exceeding 30 seconds are treated as failed.
- **Validation:** every inference output shall be checked against a fixed JSON schema before it is saved.
- **Failure handling:** on timeout or invalid output, the event is retried up to 3 times with increasing delay; after that it is logged as a processing failure and skipped.

3.1.4 Software/API Interface Requirements

- **API-01:** The system shall expose a REST API over HTTPS, using JSON for requests and responses.
- **API-02:** The API shall be grouped by resource: Auth, Users, Alerts, BlockRules, ThreatIntel, Stats, Admin.

- **API-03:** Every protected endpoint shall require a valid JWT in the Authorization header.
- **API-04:** The API shall return standard status codes: 200/201 (success), 400 (invalid input), 401 (not authenticated), 403 (not permitted), 404 (not found), 409 (conflict), 429 (too many requests), 500 (server error).
- **API-05:** Every error response shall contain a machine-readable code and a human-readable message.
- **API-06:** The system shall use a threat-intelligence provider's REST API for IP reputation lookups and shall cache results to avoid redundant calls.
- **API-07:** The system shall use an email/Slack webhook to send critical-severity notifications and shall log delivery failures.
- **API-08:** The API shall be documented using OpenAPI or a Postman collection.

3.1.5 Communication Interface Requirements

- **COM-01:** All external communication shall use HTTPS with TLS 1.2 or later.
- **COM-02:** Internal communication between the backend and the ML inference service shall use the streaming platform over a private network that is not publicly reachable.
- **COM-03:** Client requests shall time out after 30 seconds and show a retry option.
- **COM-04:** The system shall retry failed internal calls up to 3 times before reporting failure.

3.2 Functional Requirements

3.2.1 User Authentication and Authorization

- **FR-AUTH-01 (Mandatory):** The system shall allow an administrator to create analyst and administrator accounts with name, email and password.
- **FR-AUTH-02 (Mandatory):** The system shall accept only passwords of at least 8 characters with an uppercase letter, a lowercase letter and a number.
- **FR-AUTH-03 (Mandatory):** The system shall store passwords only as salted hashes (bcrypt or argon2).
- **FR-AUTH-04 (Mandatory):** The system shall authenticate users by email and password and issue a JWT access token (valid 15 minutes) and a refresh token (valid 7 days).
- **FR-AUTH-05 (Mandatory):** The system shall enforce role-based access so that Analyst and Administrator can access only their permitted pages and APIs.
- **FR-AUTH-06 (Mandatory):** The system shall let a user log out, which invalidates the refresh token.
- **FR-AUTH-07 (Mandatory):** The system shall lock an account for 15 minutes after 5 consecutive failed logins.
- **FR-AUTH-08 (Desired):** The system shall send a password reset link that is valid for 30 minutes and works once.
- **FR-AUTH-09 (Mandatory):** The system shall authenticate the WebSocket connection using the same JWT before allowing a client to receive live alert events.

3.2.2 Traffic Capture and Data Ingestion

- **FR-CAP-01 (Mandatory):** The system shall capture network packets from a live interface or replay them from a `.pcap` file.

- **FR-CAP-02 (Mandatory):** The system shall extract a 12-feature flow record from captured packets (duration, packet/byte counts each direction, mean packet length each direction, throughput, SYN/FIN flag counts, protocol).
- **FR-CAP-03 (Mandatory):** The system shall publish each extracted flow record as a JSON event to the streaming platform within 2 seconds of flow completion.
- **FR-CAP-04 (Mandatory):** The system shall record the raw flow record in the database regardless of verdict, for later analysis and model retraining.
- **FR-CAP-05 (Desired):** The system shall support pacing the replay of a `.pcap` file to simulate live traffic timing for demonstrations.
- **FR-CAP-06 (Mandatory):** If packet capture is interrupted, the system shall log the interruption and resume without losing already-published events.

3.2.3 Detection and ML Inference

- **FR-DETECT-01 (Mandatory):** The system shall score every flow event with a supervised classifier that names the attack family (or Benign).
- **FR-DETECT-02 (Mandatory):** The system shall score every flow event with an anomaly model trained only on benign traffic, independent of the supervised model.
- **FR-DETECT-03 (Mandatory, Verdict Fusion):** The system shall mark a flow malicious if the supervised model's top class is not Benign **or** the anomaly model flags it as an outlier.
- **FR-DETECT-04 (Mandatory, Explainability):** The system shall compute SHAP values for every supervised prediction and store the top 3 contributing features with each alert.
- **FR-DETECT-05 (Mandatory):** The system shall process flow events from the stream in the background and shall not block traffic capture while scoring.
- **FR-DETECT-06 (Mandatory):** If the ML inference service is unavailable, the system shall retry up to 3 times, keep events queued in the stream, and keep the rest of the platform working.
- **FR-DETECT-07 (Mandatory):** The system shall accept inference output only if it matches the defined JSON schema; otherwise it shall retry and then log the flow as a processing failure.
- **FR-DETECT-08 (Desired):** The system shall record the model version used for every verdict, to support traceability when models are retrained.

3.2.4 Severity Scoring and Explainability

- **FR-SCORE-01 (Mandatory):** The system shall assign a severity level (low, medium, high, critical) to every malicious verdict based on supervised confidence and anomaly strength.
- **FR-SCORE-02 (Mandatory):** The system shall show, for every alert, the supervised label, supervised confidence, anomaly score and the top 3 SHAP-ranked contributing features.
- **FR-SCORE-03 (Mandatory):** The system shall present the explanation in a form a non-ML-specialist analyst can read (feature name plus a bar showing contribution strength and direction).
- **FR-SCORE-04 (Desired):** The system shall let an administrator adjust the confidence threshold used to assign each severity level.
- **FR-SCORE-05 (Mandatory):** The system shall not use any field other than flow-level network features (no user identity, no payload content) in scoring, to keep the model auditable and privacy-respecting.

3.2.5 Threat Intelligence Enrichment

- **FR-INTEL-01 (Desired):** The system shall look up a suspicious source IP against a threat-intelligence provider and display its reputation score, country and last-seen date.
- **FR-INTEL-02 (Desired):** The system shall cache threat-intelligence results and reuse them for repeated lookups of the same IP within 24 hours.
- **FR-INTEL-03 (Optional):** If the threat-intelligence provider is unavailable, the system shall show the alert without enrichment rather than delaying the alert.

3.2.6 Alert Lifecycle and Analyst Response

- **FR-RESP-01 (Mandatory):** The system shall show a security analyst the live, ranked feed of alerts with severity, verdict, source/destination IP and status.
- **FR-RESP-02 (Mandatory):** The system shall let the analyst search, filter (by severity, status, source IP, date range) and sort alerts.
- **FR-RESP-03 (Mandatory):** The system shall show the flow's raw feature values alongside the ML explanation for every alert.
- **FR-RESP-04 (Mandatory):** The system shall let the analyst acknowledge, resolve or mark an alert as a false positive, with an optional reason.
- **FR-RESP-05 (Mandatory):** The system shall never block an IP automatically; a block rule is only created after explicit analyst approval.
- **FR-RESP-06 (Mandatory):** The system shall allow only valid status transitions: New → Acknowledged → Resolved / Blocked / False Positive.
- **FR-RESP-07 (Optional):** The system shall let the analyst compare 2–3 related alerts (same source IP) side by side.
- **FR-RESP-08 (Optional):** The system shall let the analyst export the current alert list as a CSV file.
- **FR-RESP-09 (Mandatory):** When the analyst approves a block, the system shall record a block rule with the IP, reason, approving analyst and timestamp.

3.2.7 Notifications

- **FR-NOT-01 (Mandatory):** The system shall push new-alert and alert-status-change events to all connected analyst dashboards in real time over WebSocket.
- **FR-NOT-02 (Desired):** The system shall send an email or Slack notification for every critical-severity alert.
- **FR-NOT-03 (Desired):** The system shall create at most one notification per event per user.
- **FR-NOT-04 (Desired):** The system shall let users mark notifications as read and turn off email notifications in Settings.

3.2.8 Dashboards and Administration

- **FR-DASH-01 (Mandatory):** The analyst dashboard shall show alert counts by severity and status, a live feed, and a rolling chart of alerts over time.
- **FR-DASH-02 (Mandatory):** The administrator dashboard shall show user counts, active block rules, pipeline health (capture, streaming, inference service status) and recent audit activity.
- **FR-ADM-01 (Desired):** The administrator shall be able to view, suspend and reactivate analyst accounts.

- **FR-ADM-02 (Desired):** The administrator shall be able to view, edit and deactivate block rules.
- **FR-ADM-03 (Desired):** The administrator shall be able to configure severity thresholds and notification recipients.
- **FR-ADM-04 (Mandatory):** The system shall provide Help Center, Terms and Conditions, Privacy Policy and About pages to all users.

3.2.9 Reporting and Analytics

- **FR-RPT-01 (Desired):** The system shall show, per time window, total flows analyzed, alerts raised, alerts by severity, and false-positive rate.
- **FR-RPT-02 (Desired):** The system shall let administrators filter analytics by date range and severity.
- **FR-RPT-03 (Optional):** The system shall let administrators export reports as CSV.
- **FR-RPT-04 (Desired):** The system shall provide a model evaluation report (accuracy, precision, recall, F1, confusion matrix, generalization gap between training and validation datasets) for the administrator and evaluators.

3.2.10 Logging, Monitoring and Audit

- **FR-LOG-01 (Mandatory, Audit Log):** The system shall record every ML verdict with the flow ID, scores, model version and timestamp.
- **FR-LOG-02 (Mandatory):** The system shall record every analyst decision (acknowledge, resolve, false positive, block approval) with the user and timestamp.
- **FR-LOG-03 (Mandatory):** The system shall record login, logout, failed login and lockout events.
- **FR-LOG-04 (Mandatory):** The system shall log errors and failures without storing passwords, tokens or raw packet payload content.
- **FR-LOG-05 (Mandatory):** The system shall provide a health-check endpoint for each component (backend, ML inference service, streaming platform).
- **FR-LOG-06 (Desired):** The system shall let the administrator view and filter audit logs, and shall not allow logs to be edited or deleted through the application.

3.3 Performance Requirements

3.3.1 Response Time

- **PERF-01:** Page loads and normal API calls (excluding ML inference) shall respond within 2 seconds for 95% of requests under normal load.
- **PERF-02:** Login shall complete within 2 seconds.
- **PERF-03:** Alert search and filter shall return within 3 seconds.

3.3.2 Processing Throughput

- **PERF-04:** The system shall be able to ingest and feature-extract at least 500 flows per minute.
- **PERF-05:** A batch of 10,000 queued flow events shall be scored within 15 minutes.

3.3.3 Concurrent Users and Requests

- **PERF-06:** The system shall support at least 50 concurrent analyst dashboard connections with no more than 1% failed requests.

- **PERF-07:** The system shall handle at least 50 API requests per second.

3.3.4 ML Processing Performance

- **PERF-08:** A single flow event shall be scored (supervised + anomaly + SHAP) within 15 seconds end to end.
- **PERF-09:** The same flow scored twice shall produce the same verdict (deterministic inference).
- **PERF-10:** The supervised classifier shall achieve at least 90% macro F1 on the CIC-IDS2017 held-out test split (target).

3.3.5 System Capacity

- **PERF-11:** The system shall store at least 1,000,000 flow records and 50,000 alerts without loss of performance.
- **PERF-12:** Storage shall be expandable without changing application code.

3.4 Logical Database Requirements

3.4.1 User Data (SQLite via Prisma)

- **User:** id, email (unique), password hash, role, created time, last login.

3.4.2 Flow and Alert Data (MongoDB)

- **Flow:** srcIp, dstIp, srcPort, dstPort, protocol, duration, packetCount, byteCount, flags, timestamp.
- **Alert:** flowId, supervisedLabel, supervisedConfidence, anomalyScore, verdict, severity, shapTopFeatures[], status, assignedToUserId, createdAt.
- **ThreatIntel:** ip, abuseScore, country, lastChecked.

3.4.3 Response and Configuration Data (SQLite via Prisma)

- **BlockRule:** id, ip, reason, alertId, createdByUserId, active, createdAt.

3.4.4 Notification and Audit Data

- **Notification:** userId, type, message, readFlag, createdAt.
- **AuditLog:** actor, action, target, details, timestamp.

3.4.5 Data Integrity and Relationships

- Email addresses shall be unique.
- Every alert shall link to an existing flow record.
- Every block rule shall link to the alert and analyst that approved it.
- Deleting a flow record shall not delete its associated audit records.
- Required fields shall be validated before saving.

3.4.6 Data Retention

- Raw flow records are retained for 90 days for model retraining, then archived or summarized.
- Alerts and audit logs are kept for at least 1 year.
- Notifications older than 90 days may be removed.

- Backups are kept for 30 days.

3.5 Design Constraints

3.5.1 Technology Constraints

- The system shall be built on the MERN stack (MongoDB, Express.js, React, Node.js).
- The ML/detection components shall be a separate service (Python, FastAPI).
- Docker shall be used for containerization, Kubernetes for final deployment.
- CI/CD, Git and Jira shall be used throughout development.
- The streaming layer shall be Kafka-API-compatible (Redpanda) so client code matches industry-standard Kafka usage.

3.5.2 Standards Compliance

- Security shall follow the OWASP Top 10.
- Communication shall follow HTTPS/TLS standards.
- Requirements documentation follows IEEE 830 / ISO/IEC/IEEE 29148.

3.6 Software System Attributes

3.6.1 Reliability

- **REL-01:** The system shall have no more than 1 critical failure per 100 hours of use during demonstration.
- **REL-02:** A failure of the ML inference service shall not stop traffic capture, login or dashboard browsing.
- **REL-03:** Failed background scoring jobs shall be retried automatically and never silently lost.

3.6.2 Availability

- **AVL-01:** The system shall be available at least 99% of the time during evaluation periods.
- **AVL-02:** After a component crash, it shall restart automatically within 2 minutes.

3.6.3 Security

- **SEC-01:** All traffic shall be encrypted with TLS, and passwords hashed.
- **SEC-02:** All inputs shall be validated and sanitized against injection (NoSQL injection, XSS).
- **SEC-03:** The system shall apply rate limiting, secure headers and restricted CORS.
- **SEC-04:** Access shall follow least privilege for every role.
- **SEC-05:** Secrets (keys, passwords) shall never be stored in the source code repository.
- **SEC-06:** Container images and dependencies shall be scanned for vulnerabilities in the CI pipeline.
- **SEC-07:** Captured flow data shall exclude payload content, storing only header-derived features, to minimize sensitive-data exposure.

3.6.4 Maintainability

- **MNT-01:** Frontend, backend API, and ML service shall be separate modules with clear interfaces.
- **MNT-02:** Code shall use linting, consistent style and meaningful commit messages.

- **MNT-03:** Automated tests shall run in CI on every push, with a target of 70% coverage for core modules.
- **MNT-04:** Logs shall be detailed enough to diagnose failures.

3.6.5 Portability

- **POR-01:** The system shall run the same in local Docker Compose and in Kubernetes, using environment-based configuration.
- **POR-02:** The frontend shall work on the latest versions of Chrome, Edge, Firefox and Safari.

3.6.6 Scalability

- **SCA-01:** The frontend, backend and ML inference service shall scale independently by adding container instances.
- **SCA-02:** Adding instances shall increase throughput without code changes.

3.6.7 Usability

- **USE-01:** A new analyst shall be able to log in and understand a live alert within 5 minutes without training.
- **USE-02:** An analyst shall be able to find the highest-severity active alert within 3 clicks from the dashboard.
- **USE-03:** The interface shall follow basic accessibility practice (contrast, labels, keyboard navigation).
- **USE-04:** Every alert's explanation shall be understandable without ML expertise.

3.7 Functional Requirement Organization

The requirements in Section 3.2 are grouped by feature area, in the order of the main system flow: access, ingestion, detection, scoring/explanation, enrichment, analyst response, notification, dashboards, reporting and audit.

3.7.1 Requirement Identification and Traceability

Each requirement has a unique ID: TYPE-AREA-NN, for example FR-DETECT-03 (functional), PERF-08, SEC-05. Each requirement is traced to a Jira epic and story, to a design element in the SDD, and to at least one test case in Appendix E.

3.7.2 Requirement Priority

- **Mandatory:** required for the system to work; built first.
- **Desired:** important; built after Mandatory in the planned phases.
- **Optional:** useful extras; built as time allows within the release.

3.7.3 Requirement Verification and Validation

Every requirement is written to be testable. Verification uses unit tests, API tests, integration tests, security tests (input validation, access control, injection), performance/load tests, and an ML evaluation on a labeled sample set. Requirements are validated by demonstrating them to the faculty and by team review.

3.8 Additional Requirements

- **ADD-01:** The system shall include help content explaining how to read an alert's severity and explanation, and how to approve a block (see FR-ADM-04).
- **ADD-02:** Source code, the Jira board and documentation shall be available to the faculty for review.
- **ADD-03:** The project shall provide a Docker Compose file to run the whole system locally, and Kubernetes manifests for final deployment.

4. Change Management Process

4.1 Change Request Process

1. Any team member, or the faculty mentor, who wants to change a requirement raises a Change Request (CR) as a Jira ticket of type "Change Request".
2. The CR records: ID, requested by, date, affected requirement IDs, description of the change, reason, and priority.
3. The team analyzes the impact of the change on the schedule, design (SDD), code and tests, and writes it in the ticket.
4. The CR is reviewed in a team meeting. Its status is set to Approved, Rejected or Deferred, with the reason recorded.
5. Approved changes are made in the SRS, and the SDD, Jira stories and test cases are updated to match.

4.2 Requirement Review and Approval

- **Reviewers:** all team members review each change request.
- **Approvers:** changes within the agreed scope are approved by team consensus. Changes that affect the scope, deliverables or timeline also need the approval of the faculty mentor, [Faculty Mentor Name].
- A change is applied only after approval, and not during a sprint that is already in progress unless it is critical.

4.3 Requirement Version Control

- The SRS is kept in a shared document with version history on, and a PDF copy is committed to the Git repository under `docs/` at each release.
- Versions use the format Major.Minor. Version 1.0 is the first baseline. Changes to requirements raise the minor number (1.1, 1.2), and a large restructuring raises the major number.
- Each version has an entry in the revision history table:

Version	Date	Author	Description of change	Approved by
1.0	20-Sep-2026	Team 8	Initial SRS	Pending

5. Document Approvals

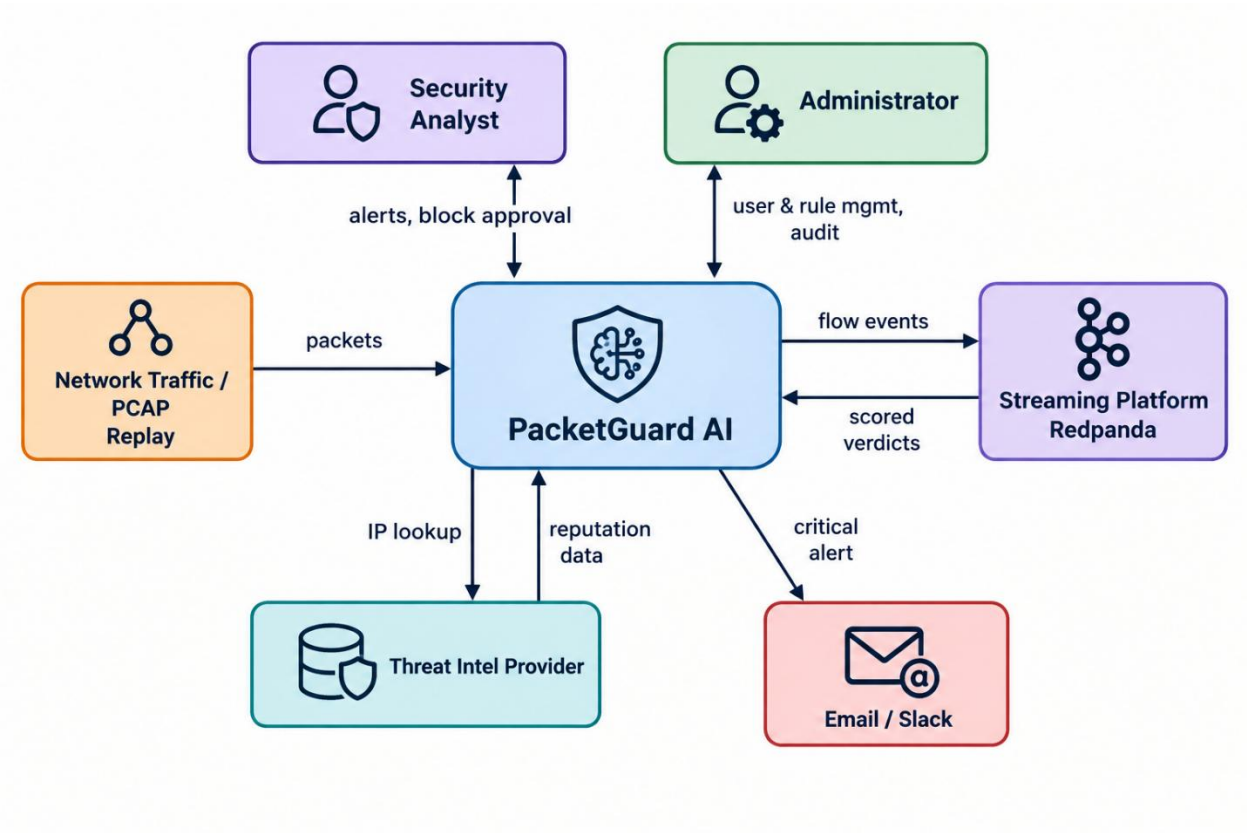
5.1 Approval and Sign-off

Name	Role	Signature	Date
Shraddha Mandal	Team Member		
Anjali Nair	Team Member		
Ayushi Nawal	Team Member		
Vinisha Shah	Team Member		

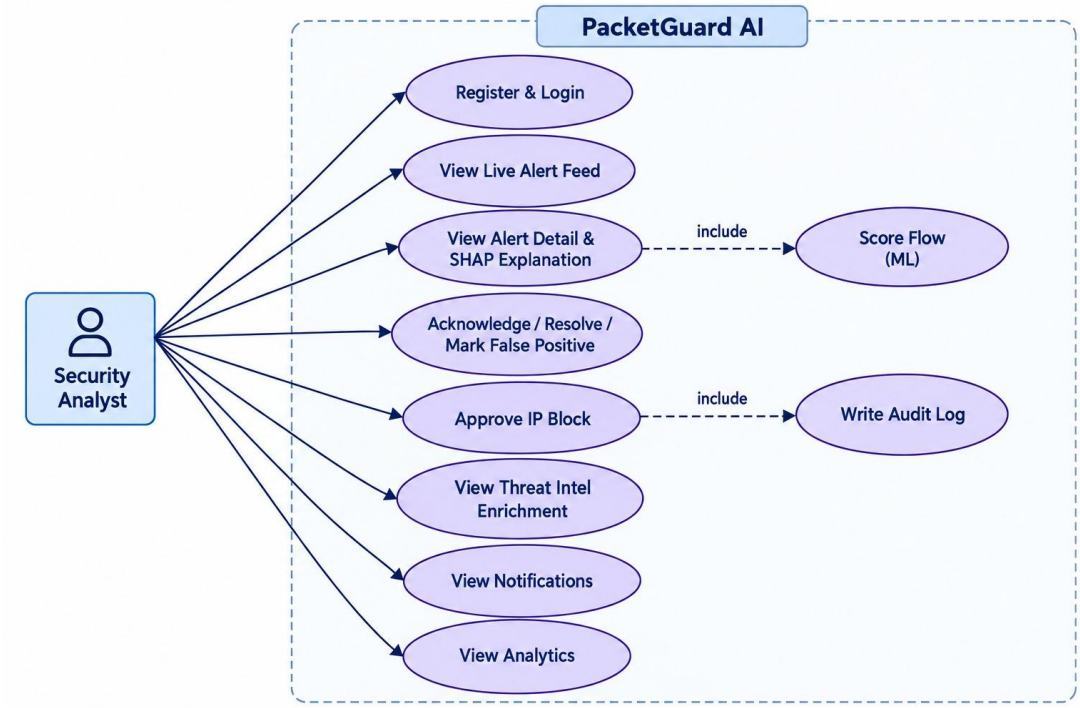
6. Supporting Information

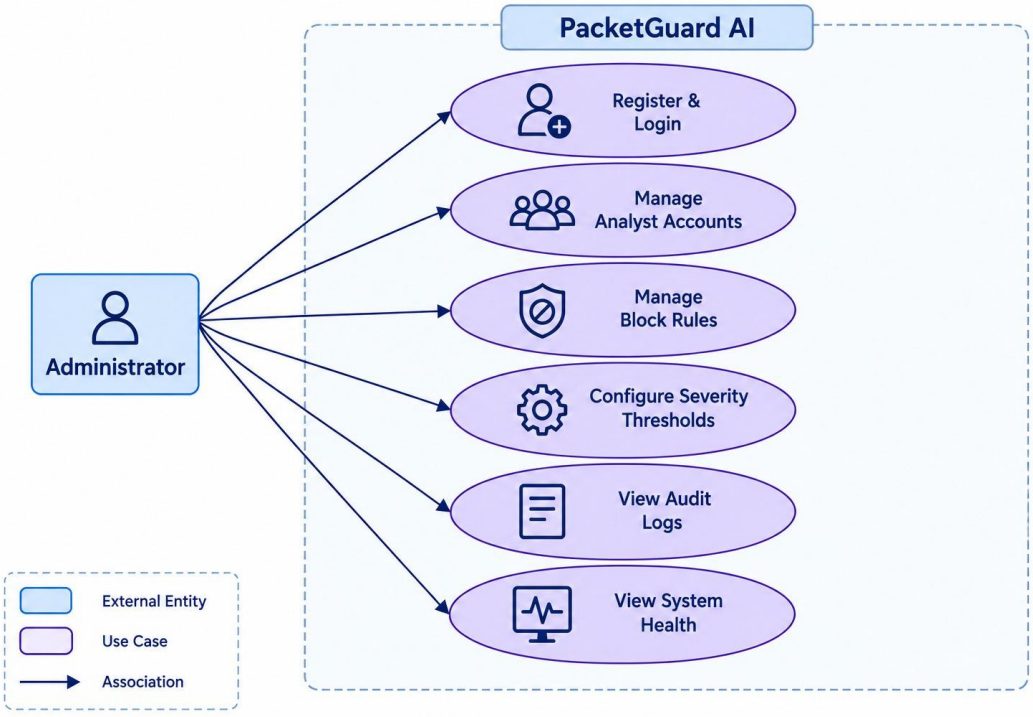
6.1 Appendix A: System Context Diagram

PacketGuard AI is shown as a black box. Analysts and administrators interact with it, and it uses the streaming platform, ML inference service, threat-intelligence provider and email/Slack.

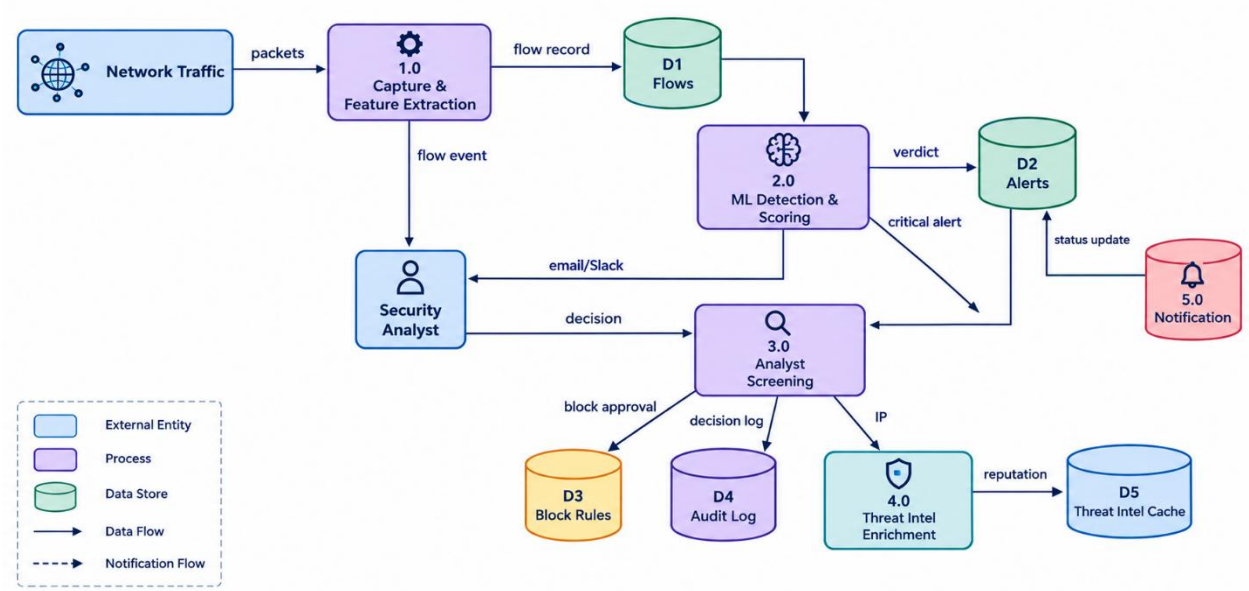


6.2 Appendix B: Use Case Diagrams

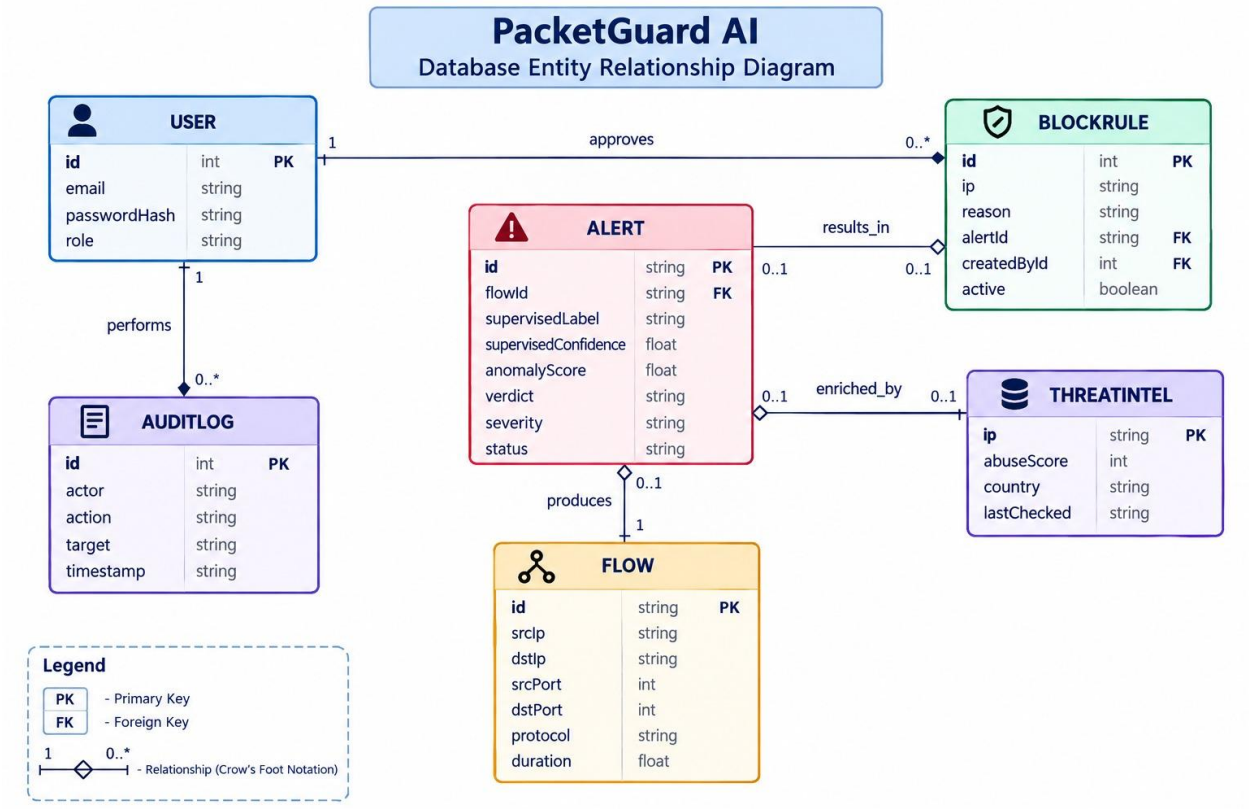




6.3 Appendix C: Data Flow Diagram (Level 1)



6.4 Appendix D: ER Diagram



6.5 Appendix E: Requirement Traceability Matrix

Requirement(s)	Requirement summary	Design element (SDD)	Test case(s)
FR-AUTH-01, 02, 03	Account creation with password rules	Auth module	TC-AUTH-01: create account with valid data. TC-AUTH-02: weak password rejected
FR-AUTH-04, 05	JWT login, RBAC	Auth module, middleware	TC-AUTH-03: login returns tokens. TC-AUTH-04: analyst blocked from admin API
FR-AUTH-07	Account lockout	Auth module	TC-AUTH-05: 5 failed logins lock the account
FR-CAP-01, 02, 03	Capture, feature extraction, streaming publish	Sniffer service	TC-CAP-01: pcap replay produces flow events. TC-CAP-02: feature schema matches training data
FR-CAP-06	Capture interruption handling	Sniffer service	TC-CAP-03: interrupted capture resumes without event loss
FR-DETECT-01, 02, 03	Supervised + anomaly + verdict fusion	ML inference service	TC-DETECT-01: known attack flow classified correctly. TC-DETECT-02: novel anomalous flow flagged by anomaly model alone
FR-DETECT-04	SHAP explainability	ML inference service	TC-DETECT-03: top-3 features returned and match manual SHAP inspection
FR-DETECT-06, 07	Failure handling, schema validation	ML inference service, streaming consumer	TC-DETECT-04: inference service down, events queue and reprocess. TC-DETECT-05: malformed output triggers retry then failure log

Requirement(s)	Requirement summary	Design element (SDD)	Test case(s)
FR-SCORE-01, 02	Severity assignment, explanation display	Backend, frontend	TC-SCORE-01: severity matches confidence thresholds. TC-SCORE-02: explanation panel renders top features
FR-INTEL-01, 02	Threat-intel lookup and caching	Threat-intel module	TC-INTEL-01: IP lookup returns reputation. TC-INTEL-02: repeated lookup within 24h uses cache
FR-RESP-04, 05, 06	Alert lifecycle, no auto-block, valid transitions	Alert module	TC-RESP-01: acknowledge/resolve work. TC-RESP-02: no block without analyst approval. TC-RESP-03: invalid status jump blocked
FR-RESP-09	Block rule creation on approval	Alert module, BlockRule model	TC-RESP-04: approved block creates a BlockRule row with correct fields
FR-NOT-01, 02	Real-time push, email/Slack for critical	Socket.io, notification module	TC-NOT-01: new alert appears live in two open dashboard tabs. TC-NOT-02: critical alert sends an email/Slack message
FR-DASH-01, 02	Analyst and admin dashboards	Frontend, stats API	TC-DASH-01: dashboard counts match database
FR-ADM-01, 02, 03	Admin functions	Admin module	TC-ADM-01: suspend user blocks login. TC-ADM-02: block rule edit/deactivate works. TC-ADM-03: threshold change affects new severities
FR-LOG-01, 02, 03	Audit logging	Audit module	TC-LOG-01: every ML

Requirement(s)	Requirement summary	Design element (SDD)	Test case(s)
			verdict logged. TC-LOG-02: every analyst decision logged. TC-LOG-03: logs cannot be edited through the app
PERF-01 to 10	Performance targets	All modules	TC-PERF-01: load test with 50 concurrent dashboards. TC-PERF-02: flow scored within 15 seconds. TC-PERF-03: verdict determinism over repeated runs
SEC-01 to 07	Security requirements	All modules, CI	TC-SEC-01: NoSQL injection and XSS attempts fail. TC-SEC-02: rate limiting triggers. TC-SEC-03: container image scan has no critical findings
REL-02, AVL-02, POR-01, SCA-01	Resilience, portability, scaling	Docker, Kubernetes	TC-DEP-01: stop ML service, rest of platform still works. TC-DEP-02: crashed container restarts. TC-DEP-03: same behavior in Compose and Kubernetes
USE-01, 02	Usability	Frontend	TC-USE-01: new analyst understands an alert within 5 minutes. TC-USE-02: highest-severity alert found in 3 clicks

6.6 Appendix F: Sample API / Input-Output Formats

Login (`POST /api/auth/login`)

```
{ "email": "analyst@example.com", "password": "Str0ngPass1" }
```

Response 200:

```
{
  "accessToken": "<jwt>",
  "refreshToken": "<token>",
  "expiresIn": 900,
  "user": { "id": 1, "role": "analyst" }
}
```

List alerts (`GET /api/alerts?severity=critical&status=new`)

```
{
  "alerts": [
    {
      "id": "a_9001",
      "srcIp": "203.0.113.55",
      "dstIp": "10.0.0.12",
      "supervisedLabel": "DDoS",
      "supervisedConfidence": 0.94,
      "anomalyScore": -0.31,
      "verdict": "malicious",
      "severity": "critical",
      "shapTopFeatures": [
        { "feature": "flow_packets_per_sec", "contribution": 0.42 },
        { "feature": "syn_flag_count", "contribution": 0.31 }
      ],
      "status": "new",
      "createdAt": "2026-09-20T10:15:00Z"
    }
  ],
  "note": "Severity and verdict are recommendations. The final block decision belongs to the analyst."
}
```

Update alert status (`PATCH /api/alerts/:id/status`)

```
{ "status": "acknowledged" }
```

Approve a block (`POST /api/alerts/:id/block`)

```
{ "reason": "Confirmed DDoS source, high confidence and repeated pattern" }
```

Response 201:

```
{ "blockRuleId": 501, "ip": "203.0.113.55", "active": true }
```

Error format (all endpoints)

```
{
  "error": {
    "code": "INVALID_STATUS_TRANSITION",
    "message": "Cannot move an alert from 'resolved' back to 'new'."
  }
}
```